

Focused File Guide

This guide focuses on the MATLAB control, analysis, UI, and EE268 workflow files; URDF, meshes, ROS config, and ROS launch files are intentionally not described because they are not the part you need to study for the MATLAB workflow.

What To Study First

Start with `doritos_matlab/main_files/`, then `doritos_matlab/ee268_analysis/ee268_master_ui.m`, then `doritos_matlab/ee268_analysis/run_ee268_analysis.m`, and only after that look at the Simulink bridge scripts.

Root Folder

- `.gitignore` controls what Git ignores, and it does that by listing generated MATLAB/Simulink folders, cache files, PNG plots, logs, autosaves, and temporary outputs.
- `README.md` gives the project quick start, and it does that by listing the required MATLAB paths plus the main commands for analysis, testing, UI launch, and Simulink loading.
- `PROJECT_MAP.md` gives a short learning map, and it does that by grouping the project into core math, EE268 UI/analysis, Simulink model, trajectory data, and generated files.
- `FILE_BY_FILE_GUIDE.md` is this focused guide, and it does that by explaining the MATLAB-side files you actually need to understand.
- `FILE_BY_FILE_GUIDE.pdf` is the PDF version of this guide, and it is generated from the Markdown file for easier reading or printing.
- `MatlabModelDocumentation.pdf` is the original documentation, and it explains the model through a PDF report rather than executable source code.

`doritos\_matlab/main\_files/` - Core Delta Robot Math

- `delta_params.m` defines the robot geometry, limits, home position, and new hardware command convention, and it does that by returning one shared `params` struct used by every FK/IK and analysis function.
- `delta_command_to_model.m` converts user/hardware command angles into internal model angles, and it does that by applying the configured `90 deg` zero reference and the flipped sign direction.
- `delta_model_to_command.m` converts internal model angles back into user/hardware command angles, and it does that by reversing the zero offset and sign flip from `delta_command_to_model.m`.
- `delta_IK.m` computes inverse kinematics from Cartesian position to joint angles, and it does that by projecting the target into each arm plane, solving the closed-form angle equation, then checking reachability and joint limits.
- `delta_FK.m` computes forward kinematics from joint angles to Cartesian position, and it does that by building three shifted elbow-center spheres, subtracting sphere equations into a linear system, then solving the remaining quadratic in `z`.
- `delta_singularity.m` checks singularity risk, and it does that by computing the FK pose, forearm unit vectors, joint and Cartesian Jacobians, determinant/condition metrics, and type-1/type-2 singularity flags.
- `delta_workspace.m` finds the reachable workspace, and it does that by scanning a 3D bounding box or checking a single point with `delta_IK`.
- `delta_manipulability_space.m` evaluates workspace quality, and it does that by sampling points, running IK, calculating Jacobian-based manipulability and conditioning, and optionally plotting/saving the results.

`doritos\_matlab/` - MATLAB Entry Points And Test Scripts

- `run_delta_analysis.m` is the general one-command analysis runner, and it does that by calling workspace sampling, manipulability analysis, and home singularity checks using the current `delta_params`.
- `test_fk_ik_consistency.m` verifies the kinematic math, and it does that by sending known reachable points through IK, reconstructing them with FK, and printing the reconstruction error.
- `plot_workspace_surface.m` creates a visual reachable-workspace cloud, and it does that by sampling points, filtering valid IK solutions, coloring optional quality metrics, and saving the plot/data when requested.
- `generate_circle_signal_builder.m` creates a circle trajectory dataset, and it does that by defining waypoints, interpolating them into dense samples, validating each sample with IK/singularity checks, and saving a Signal Editor scenario.

- `signal_builder_circle_test.mat` stores the reusable circle test trajectory, and it does that by saving timeseries and `Simulink.SimulationData.Dataset` variables generated by `generate_circle_signal_builder.m`.
- `Delta_LiveScript.mlx` is an interactive MATLAB Live Script, and it does that by mixing executable MATLAB cells with formatted notes for exploratory analysis.
- `delta_analysis_report.mat` is a generated output from analysis, and it does that by storing the `report` struct produced when `run_delta_analysis.m` saves results.

`doritos\_matlab/ee268\_analysis/` - Main UI, EE268 Analysis, And Simulink Bridge

- `ee268_master_ui.m` is the main app you use, and it does that by loading EE268 workspace results, drawing the reachable cloud and robot overlay, validating moves, publishing `x_ref_ts/y_ref_ts/z_ref_ts`, and optionally running the Simulink model.
- `run_ee268_analysis.m` builds the EE268 workspace used by the UI, and it does that by computing the tool offset, sampling EE positions, evaluating IK/manipulability/singularity metrics, saving `.mat` results, and regenerating plots.
- `configure_doritos_mk2_ui_bridge.m` updates the Simulink Kinematics bridge, and it does that by adding From Workspace blocks, source switches, startup home references, model callbacks, and the MATLAB Function block IK script.
- `configure_doritos_mk2_pen_contact.m` adds the pen/contact visualization setup, and it does that by inserting Simscape physical ports, rigid transforms, brick solids, transform sensors, and logging blocks into the model.
- `ee268_pick_place_signal.mat` stores a generated pick-and-place scenario, and it does that by saving arm reference signals, gripper/contact references, rigid-transform notes, and a Simulink scenario dataset.
- `ee268_analysis_results.mat` is the generated data file the UI depends on, and it does that by storing sampled reachable EE points, platform points, joint angles, manipulability, conditioning, thresholds, and context.
- `ee268_context.mat` stores the EE268 reference context, and it does that by saving the platform home, physical EE home, tool offset, workspace limits, and setup assumptions.
- `test_ee268_master_ui_options.m` smoke-tests the UI, and it does that by opening the UI invisibly, pressing major buttons programmatically, and recording pass/fail results.
- `inspect_doritos_mk2_model.m` is a model inspection helper, and it does that by loading the Simulink model and reporting useful block/model structure for debugging.
- `inspect_kinematics_blocks.m` is a Kinematics subsystem inspection helper, and it does that by loading the model and listing blocks inside the Kinematics area.
- `inspect_kinematics_layout.m` is a Kinematics layout inspection helper, and it does that by reading block positions so bridge scripts can place new blocks cleanly.
- `doritos_mk2_ui_bridge_report.txt` is a generated bridge report, and it does that by recording the blocks and signal lines after `configure_doritos_mk2_ui_bridge.m` modifies the model.
- `ui_option_test.log` is a generated UI test log, and it does that by storing detailed output from running UI option tests.
- `ui_option_test_summary.txt` is a generated UI test summary, and it does that by condensing the UI test results into a short text report.
- `ui_test_results.mat` is a generated UI test result file, and it does that by storing the test result struct from `test_ee268_master_ui_options.m`.

`doritos\_mk2/` - Simulink Model Folder Summary

- `doritos_mk2/` contains the physical robot model and export assets, and for the MATLAB workflow the file that matters most is `doritos_mk2.slx` because the UI bridge publishes references into its Kinematics subsystem.
- `doritos_mk2/doritos_mk2.slx` is the main Simulink/Simscape model, and it works by receiving EE reference signals, converting them through the Kinematics MATLAB Function block, and driving the simulated robot assembly.
- `doritos_mk2` also contains URDF, mesh, ROS config, ROS launch, backup, and cache files, and those are intentionally not explained file-by-file here because your current work is MATLAB/Simulink control rather than ROS model export.

`writing.mat/` - Writing Trajectory Data

- `AHMED_scenario.mat` stores the final AHMED writing path, and it does that by keeping a Simulink scenario dataset that the master UI can load as `x/y/z` references.

- `BOODY_scenario.mat` stores the final BOODY writing path, and it does that by keeping a Simulink scenario dataset that the master UI can load as `x/y/z` references.
- `HAMZA_scenario.mat` stores the final HAMZA writing path, and it does that by keeping a Simulink scenario dataset that the master UI can load as `x/y/z` references.
- `NADA_scenario.mat` stores the final NADA writing path, and it does that by keeping a Simulink scenario dataset that the master UI can load as `x/y/z` references.
- `POWER_scenario.mat` stores the final POWER writing path, and it does that by keeping a Simulink scenario dataset that the master UI can load as `x/y/z` references.
- `SH7S_scenario.mat` stores the final SH7S writing path, and it does that by keeping a Simulink scenario dataset that the master UI can load as `x/y/z` references.

`writing.mat/files/` - Raw Writing Signal Generation

- `generate_robot_signals.py` generates raw writing signals, and it does that by defining letter strokes, interpolating drawing and pen-lift moves, and saving word paths into `.mat` files.
- `convert_signal_mats_to_scenarios.m` converts raw signal `.mat` files into UI-ready scenarios, and it does that by extracting time/X/Y/Z vectors and packing them into `Simulink.SimulationData.Dataset` objects.
- `AHMED_signal.mat` is the raw/generated AHMED signal file, and it does that by storing intermediate signal data before or alongside scenario conversion.
- `BOODY_signal.mat` is the raw/generated BOODY signal file, and it does that by storing X/Y/Z/time vectors plus scenario data.
- `HAMZA_signal.mat` is the raw/generated HAMZA signal file, and it does that by storing X/Y/Z/time vectors plus scenario data.
- `NADA_signal.mat` is the raw/generated NADA signal file, and it does that by storing X/Y/Z/time vectors plus scenario data.
- `POWER_signal.mat` is the raw/generated POWER signal file, and it does that by storing X/Y/Z/time vectors plus scenario data.
- `SH7S_signal.mat` is the raw/generated SH7S signal file, and it does that by storing X/Y/Z/time vectors plus scenario data.
- `AHMED_signal_check.txt` and `ahmed_mat_contents.txt` are ignored inspection notes, and they do that by recording quick checks of raw AHMED signal contents.

Ignored Generated Files You Do Not Need To Study

- PNG plots are ignored now, and they are generated visual outputs rather than source logic.
- `slprj/`, `tmp_slx/`, and `*.slxc` are Simulink generated cache/build outputs, and MATLAB recreates them when needed.
- `*.log`, `*_output.txt`, `*_summary.txt`, and old inspection dumps are ignored run artifacts, and they only preserve previous console output or debugging notes.
- backup files such as `doritos_mk2_1.slx` and `doritos_mk2_2.slx` are ignored old model copies, and they are not the primary model you run.